

Lite3 Pilot — joystick movement logic & full command catalog

Complete, implementation-accurate reference for the web console (port 8123) and the Python SDK. All values are the live defaults in config.json.

1. Joystick movement logic

1.1 Inputs

Input	Mapping
Left pad (labels FWD / turn / BACK)	up/down = forward/back (vx) · left/right = turn (wz)
Right pad (labels FWD / slide / BACK)	left/right = slide sideways (vy) · up/down = forward/back — the stronger of the two pads wins for vx
↵ Slide left / ➞ Slide right buttons	hold to side-walk at 0.4 m/s × speed-scale (sends lateral velocity; in Retroid mode the side axis of the 42-byte frame)
Keyboard	W/S or ↑/↓ = forward/back · A/D or ←/→ = turn · Space (hold) = E-STOP

Pads and keys can be mixed; diagonals combine axes (e.g. forward + slide).

1.2 Per-axis processing (identical for both pads)

1. **Vector from pointer:** each pad returns x/y in $-1 \dots +1$ relative to its centre.
2. **Deadzone:** magnitude below **0.08** = neutral (no command).
3. **Normalisation:** the magnitude past the deadzone is rescaled so small pushes still count: $s = (m - 0.08) / (1 - 0.08)$, clamped to 1.
4. **Stick curve:** $s = s^{\text{curve}}$ when curve > 1 (default **1.25**, “Soft”) — finer control near the centre.
5. **Speed scale:** multiply by the scale setting (50 / 75 / **100** / 150 %).
6. **Axis assignment:** $vx = -y \times s \times \text{scale}$ · $wz = x \times s \times \text{scale}$ (left pad) · $vy = x \times s \times \text{scale}$ (right pad).
7. **Reverse-steer mirror:** when reversing ($vx < 0$) the steering sign is flipped, so “left” still turns the robot left while backing up.
8. **Clamps (server side):** $vx \leq 0.5 \text{ m/s}$ · $vy \leq 0.4 \text{ m/s}$ · $wz \leq 0.8 \text{ rad/s}$ · per-axis **sign** (Forward / Turn / Lateral) applied on send.

1.3 Sending

- Commands are produced by a **60 ms** loop; additionally a drive update fires immediately on press and on every move event (zero-latency response).
- The engine streams at up to **50 Hz** (rate_hz) while the request is fresh, keeping the robot in auto mode (0x21010C03 interleaved) with a heartbeat (0x21040001) every **300 ms**.
- **Tail:** the last velocity is kept alive for **300 ms** after the last input, then zeroed; when a velocity stream ends the engine also fires the **official brake frames** (0x21010C0B, values 0 and 1) for ~0.35 s so the robot cannot coast.

1.4 Release & stopping

- Pads are **spring-centred**: on release the knob animates home (160 ms) and the robot stops — a stop command is sent immediately, not on the next tick.
- Release anywhere (window-level capture), losing window focus, hiding the tab, **STOP**, or **E-STOP** all stop the robot.
- **E-STOP** latches: every command is refused and stop frames keep being sent until ↻ **Return**; a latched E-stop also cancels a running action.

1.5 Drive modes (Advanced → Joystick mode)

- **Velocity frames (proven)** — sends 0x140 (x), 0x145 (y), 0x141 (yaw), each type 1 with one **f64** payload (value field 8), plus the auto-mode frame.
- **Retroid / official SDK** — sends the same 42-byte frames as the physical remote to UDP **12121**: header 0x55 0x66, ctrl, data_len 32, seq, id 1, checksum (byte-sum of payload); payload = 10 × u16 buttons + 4 × i16 axes (left_x, left_y, right_x, right_y) + 2 × u16 axis buttons; range ±1000. Mapping: left_y = forward, right_x = yaw (sign configurable), left_x = slide. The robot's own controller then interprets the sticks.

1.6 Velocity semantics

- Velocity commands are honoured **only while the robot is standing** (in auto mode); while sitting/lying they are ignored silently.
- UDP gives **no acknowledgement**; confirm motion visually or via telemetry.

2. Command catalog (identical to the app buttons)

2.1 Built-in top-row buttons

Button	Command	Code	Behaviour
 STAND	posture toggle	0x21010202	One code for both poses: from sitting it stands; if already standing it sits (state-tracked, skipped when already there)
 SIT	posture toggle	0x21010202	Same toggle, aimed at the sitting pose
 HELLO	hello pose	0x21010506	Sits first if needed, then replays the pose 3× at ~1 Hz (verified on the robot)
 DANCE	dance / moonwalk	0x2101030C	Standing pose; verified
 TURN OVER	turn over	0x21010205	Rolls the robot onto its back; forces the sitting pose first; slow replay

2.2 Recovery block (from upside down)

Button	Command	Code	Behaviour

 Recover left /  Recover right	stand toggle used as recovery	0x21010202	Replays the stand toggle at 1 Hz to bring the robot back to its feet (proved on the robot)
--	-------------------------------	------------	--

2.3 Poses & actions

Button	Command name	Code	Required pose
 Twist	twist	0x21010204	standing
 Twist jump	twist_jump	0x2101020D	standing (forced)
 Backflip	backflip	0x21010502	sitting (forced)
 Long jump	long_jump	0x2101050B	sitting (forced)
 Hello /  Dance /  Turn over	hello / dance / turn_over	0x21010506 / 0x2101030C / 0x21010205	sit / stand / sit (forced)

2.4 Modes

Button	Command name	Code	Behaviour
 Manual mode	mode_manual	0x21010C02	single-shot switch (remote/manual mode)
 Move mode	mode_move	0x21010D06	single-shot switch (move mode)

2.5 Gaits

Button	Command name	Code	Behaviour
 Slow	gait_slow	0x21010300	single-shot gait switch
 Medium	gait_medium	0x21010307	single-shot gait switch
 Fast	gait_fast	0x21010303	single-shot gait switch
 Crawl	gait_crawl	0x21010406	single-shot gait switch

2.6 Execution rules that apply to every command

- **Posture handling:** an action declares its required pose; the console re-poses only when the tracked state is known to differ, or always when force_posture is set (rolls, jumps, hello). With an unknown state and no force, the action is sent directly (press STAND/SIT first if the robot is in the other pose).
- **Replay:** poses/rolls are re-sent 3× at ~1 Hz (“slow replay”); modes/gaits are single-shot; other frames use a short burst.
- **Cancellation:** E-STOP aborts any action mid-play.
- **Safety:** keep the area clear and the physical E-stop within reach for any moving command.

Sources: DeepRobotics repositories and manuals, community protocol tables, and live verification on the robot (hello, dance, stand/sit, recovery, velocity driving, camera). Codes marked “forced/logged” but not yet verified on this unit should be confirmed on a stand before regular use.